Open Access | https://doi.org/10.25080/tukr4275



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Numerical Stability Pitfalls in Scientific Optimization Pipelines

Aayush Gauba[1]

[1]Student and Researcher

Abstract

Optimization algorithms are central to scientific machine-learning and parameter-estimation workflows. Yet even mature scientific Python workflows can hide numerical instability: an optimizer may satisfy its stopping criterion while returning parameters that are poorly conditioned, physically implausible, or highly sensitive to small perturbations. In a parameter-estimation workflow, for example, apparently converged coefficients may still change sharply after tiny perturbations to the data.

This paper examines stability pitfalls in scientific machine-learning and parameter-estimation workflows built around SciPy optimization routines. It analyzes how ill-conditioned objectives, poor parameter scaling, unstable finite-difference approximations, floating-point overflow, nearly singular Hessians, and initialization sensitivity can destabilize an otherwise routine optimization run. The emphasis is practical rather than algorithmic: the goal is to help researchers recognize failure modes that are not visible from a convergence flag alone.

Each failure mode is linked to concrete diagnostics, including condition-number analysis, gradient-norm monitoring, Hessian-spectrum inspection, perturbation tests, and convergence-log inspection. A controlled instability stress test on the HIGGS workflow further demonstrates how severe conditioning pathologies can induce optimizer-status reliability mismatches and catastrophic gradient instability. The paper then discusses mitigation strategies such as nondimensionalization, regularization, trust-region methods, analytic derivatives, parameter transformations, and multi-start validation.

Keywords numerical stability, scientific optimization, SciPy, conditioning, numerical auditing, HIGGS

Published Jul 13, 2026

Correspondence to
Aayush Gauba
gauba.aayush@gmail.com

Open Access

Copyright © 2026 Gauba. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

Optimization is a routine component of scientific machine-learning and parameter-estimation workflows, including statistical inference, calibration, and data-fitting pipelines.

The SciPy ecosystem provides a broad collection of algorithms that have become standard tools in the scientific Python community. Functions such as `scipy.optimize.minimize`, `least_squares`, and `curve_fit` are frequently used in both research and industrial systems.

However, optimization routines can fail in subtle ways: a solution may appear mathematically valid while remaining highly sensitive to perturbations, physically meaningless, or numerically unstable. Common causes include ill-conditioned landscapes, floating-point limitations, singular Jacobians, unstable gradients, poor scaling, and erratic optimization dynamics.

This paper investigates these failures at the workflow level. Rather than proposing new algorithms, it studies how existing SciPy-based pipelines fail and how those failures can be diagnosed through residual scale, gradient norm, step history, condition number,

perturbation sensitivity, and consistency across algorithmic choices. The central thesis is that reliability often depends more strongly on conditioning, preprocessing, and numerical auditing than on optimizer choice alone.

The paper makes three practical contributions. First, it organizes common optimizer failures into a taxonomy of geometric, arithmetic, differential, and structural instability. Second, it presents a large-scale empirical study of SciPy optimizers on the University of California, Irvine (UCI) HIGGS dataset, emphasizing preprocessing, Hessian conditioning, and computational cost. Third, it synthesizes these observations into a workflow-oriented auditing protocol for researchers who use optimization inside larger scientific workflows.

The discussion complements classical numerical analysis texts and software documentation rather than replacing them. References such as J. Nocedal and S. J. Wright [1] and N. J. Higham [2] provide rigorous treatments of conditioning, floating-point arithmetic, and convergence theory. The present work addresses a different gap: the distance between algorithmic theory and the practical behavior of end-to-end scientific Python workflows. Standard references explain what can go wrong in principle, but they do not show how reliably these failures appear across randomized runs in SciPy pipelines or package the resulting checks as a staged auditing protocol for working researchers. The contribution here is therefore organizational and empirical rather than theoretical: it connects instability mechanisms to observable diagnostic signatures, quantifies their frequency under controlled stress conditions, and assembles the guidance into a reproducible protocol targeted at the scientific Python ecosystem.

2. Optimization in Scientific Computing

Before examining specific failure modes, it is useful to establish the computational setting in which they arise. Most scientific optimization pipelines can be viewed as numerical procedures for solving problems of the form:

$$\min_{x \in \mathbb{R}^n} f(x) \tag{1}$$

where $f(x)$ represents an objective function whose evaluation may itself involve simulation, statistical estimation, numerical integration, or data fitting. In practice, such objectives often originate from least-squares fitting, maximum likelihood estimation, energy minimization, residual minimization, or variational formulations.

These objectives are rarely solved in isolation. They are usually embedded in software workflows that choose an algorithm, evaluate the objective repeatedly, estimate derivatives, and decide when convergence has occurred. SciPy provides several interfaces for this process.

2.1. Common Optimization Interfaces

SciPy exposes these workflows through routines such as `minimize`, `least_squares`, `curve_fit`, `root`, `differential_evolution`, and `basinhopping`. These interfaces include methods such as BFGS, L-BFGS-B, Nelder-Mead, Powell, conjugate gradient, trust-region methods, and Levenberg-Marquardt. Table 1 summarizes several common warning signs and first-line responses.

Table 1. *Common numerical failure modes in optimization pipelines and practical first-line responses.*

Failure mode	Observable symptom	Typical response
Ill-conditioning	Narrow valleys, large condition numbers, or zig-zagging iterates	Rescale variables or regularize

Finite-difference noise	Erratic gradients	Supply analytic or complex-step derivatives
Overflow or underflow	Infinite, zero, or NaN objective values	Reformulate in log space
Nearly singular curvature	Flat directions, unstable Newton steps, or covariance blow-up	Use damping or trust regions
Initialization sensitivity	Different runs give different optima	Use multi-start analysis

3. Conditioning and Numerical Sensitivity

Conditioning measures how strongly a numerical problem amplifies perturbations. In optimization, it determines whether small errors in data, arithmetic, or intermediate calculations remain harmless or become large changes in the returned solution.

For a nonsingular matrix A, the normwise condition number is defined as [3], [4]:

$$\kappa(A) = \mid A \mid \cdot \mid A^{-1} \mid \tag{2}$$

Large condition numbers imply that small input perturbations can produce large solution changes. In optimization, this sensitivity often appears as distorted curvature, unreliable descent directions, or fitted parameters that shift substantially under tiny data changes.

3.1. Geometric Interpretation

Ill-conditioned optimization problems often contain narrow valleys or highly anisotropic curvature. Optimizer iterates may oscillate or converge extremely slowly.

The Rosenbrock function, $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, is a compact toy example: it has a simple global minimum at $(1, 1)$, but its curved valley can still slow or destabilize optimization. It is used here only to illustrate geometry; the HIGGS experiment later in the paper provides the empirical evidence.

4. Ill-Conditioned Objective Functions

Ill-conditioned objectives can force an optimizer to spend many iterations correcting direction rather than making meaningful progress toward the minimizer. Small parameter perturbations then produce disproportionate changes in gradients or curvature, making the returned parameters fragile.

4.1. Characteristics of Ill-Conditioned Systems

Ill-conditioned optimization problems commonly exhibit:

- Slow convergence
- Erratic optimization dynamics
- High sensitivity to initialization
- Large curvature imbalance
- False convergence
- Oscillatory updates

4.2. Practical Consequences

These numerical symptoms matter because they directly affect scientific interpretation. In applications, ill-conditioning can produce:

- Unstable parameter estimation
- Non-physical fitted states
- Unstable downstream model evaluations
- Unreliable confidence intervals

4.3. Polynomial Basis Conditioning

Polynomial regression using Vandermonde feature construction provides a controlled example of geometric instability in scientific optimization workflows. As polynomial degree increases, the resulting design matrix becomes progressively ill-conditioned, amplifying sensitivity to perturbation and destabilizing least-squares estimation.

To quantify this effect, Vandermonde matrices were constructed across 100 random uniformly sampled grids on the interval $[0, 1]$, with 100 sample points in each grid. Table 2 reports the resulting condition-number statistics as polynomial degree increases.

The results demonstrate rapid instability amplification under increasing polynomial complexity. By degree 20, condition numbers frequently exceeded 10^{15}, entering regimes associated with catastrophic numerical sensitivity and effectively singular optimization geometry.

Table 2. *Condition-number growth in Vandermonde polynomial feature matrices across 100 random uniformly sampled grids on the interval* $[0, 1]$.

Polynomial Degree	Mean Condition Number	Std Condition Number	Median Condition Number
5	4.156×10^{3}	5.111×10^{2}	4.043×10^{3}
10	2.844×10^{7}	7.989×10^{6}	2.612×10^{7}
15	2.239×10^{11}	1.274×10^{11}	1.908×10^{11}
20	1.975×10^{15}	2.065×10^{15}	1.455×10^{15}

5. Parameter Scaling Problems

Poor parameter scaling often compounds ill-conditioning. Many algorithms behave best when parameter dimensions have comparable numerical scales, but scientific systems often contain parameters spanning many orders of magnitude.

Consider:

$$\theta = [10^{-12}, 10^{6}] \tag{3}$$

This imbalance destabilizes gradient-based optimization because a step size that is reasonable for one coordinate may be negligible or destructive for another.

5.1. Effects of Poor Scaling

Poor parameter scaling may cause:

- Vanishing updates
- Exploding optimization steps
- Numerical cancellation
- Loss of precision
- Extremely slow convergence

5.2. Search Geometry Distortion

Poor scaling distorts the geometry of optimization landscapes. Gradient-based methods may spend substantial effort compensating for coordinate scale rather than modeling the true curvature of the objective, while quasi-Newton approximations can become dominated by scale artifacts.

5.3. Mitigation Techniques

For this reason, scaling should be treated as part of the model formulation rather than as a cosmetic preprocessing step. Standardization is useful when parameters have empirical units and comparable statistical variation; nondimensionalization is preferable when physical scales are known; min-max normalization is useful for bounded parameters; logarithmic transformations stabilize positive parameters spanning orders of magnitude; and whitening can reduce correlations in locally linear least-squares problems. Bounds in methods such as L-BFGS-B should be chosen after scaling, because poorly scaled bounds can still distort the effective search geometry.

6. Finite Difference Instability

Even when the objective is well scaled, instability can enter through derivative estimation. Many SciPy routines estimate gradients numerically when analytic derivatives are not supplied.

Forward finite differences are commonly used:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h} \tag{4}$$

Choosing a stable step size h is difficult because the approximation must balance two competing error sources [5]. SciPy finite-difference settings are often reasonable for well-scaled variables, but they can fail when parameters are extremely small, extremely large, or measured in incompatible units.

6.1. Roundoff Error

When h becomes extremely small, floating-point subtraction causes catastrophic cancellation. The two function values become nearly identical in floating-point representation, so their difference may contain mostly roundoff error rather than derivative information.

$$f(x+h) - f(x) \tag{5}$$

may lose significant numerical precision.

6.2. Truncation Error

When h becomes too large, the approximation no longer accurately represents the local derivative. The finite-difference quotient then reflects curvature over a broad interval rather than the slope at the point of interest.

Together, roundoff and truncation error show why default finite-difference settings should not be treated as universally reliable. Large step sizes reflect nonlocal curvature, while extremely small step sizes suffer from catastrophic cancellation and floating-point roundoff error; the stable operating region is problem- and scale-dependent. When the objective permits it, complex-step differentiation or automatic differentiation can provide more stable derivative information than real-valued finite differences.

7. Floating-Point Precision Limitations

Finite-difference errors are one manifestation of a more general constraint: optimization pipelines are fundamentally limited by finite precision arithmetic. Double precision floating-point values cannot represent arbitrary real numbers exactly, and repeated objective evaluations can accumulate or amplify these small representation errors.

7.1. Precision Considerations

Floating-point format can directly affect optimizer behavior. Reduced precision may erase small objective differences or gradient information, especially in narrow valleys or near flat residual surfaces. In such cases, convergence failure is not caused by the optimization algorithm alone; it is caused by insufficient numerical resolution in the objective evaluation.

The practical implication is that precision should be treated as an experimental parameter. Researchers should record the arithmetic precision used in objective evaluations and should be cautious when using reduced precision for optimization problems with narrow valleys, small residuals, or ill-conditioned curvature.

7.2. Overflow

Exponential functions provide a simple illustration: evaluating $\exp(1000)$ in double precision overflows, replacing a finite mathematical quantity with an infinite floating-point value. In optimization, this can erase meaningful search directions unless the objective is reformulated, for example in log space.

7.3. Underflow

Small probabilities may collapse to zero during likelihood optimization. Once this happens, log-likelihoods, gradients, or likelihood ratios can become undefined even though the underlying mathematical expression is finite.

7.4. Loss of Significance

Subtraction between nearly equal numbers causes catastrophic cancellation, which is especially damaging in residual computations, finite differences, and objective functions formed from differences of large quantities.

7.5. Stable Reformulations

Stable reformulations reduce these risks by changing the numerical representation without changing the mathematical quantity being computed. For example, the log-sum-exp trick improves stability:

$$\log \sum_i e^{x_i} = m + \log \sum_i e^{x_i - m} \tag{6}$$

where:

$$m = \max_i x_i \tag{7}$$

8. Singular Hessians and Curvature Instability

The same sensitivity appears in second-order information. Hessian matrices encode local curvature and are often used to construct Newton steps, covariance approximations, or uncertainty estimates.

The Hessian matrix is:

$$H_{ij} = [\nabla^2 f]_{ij} \tag{8}$$

When Hessians become singular or nearly singular, optimization becomes unstable because the local quadratic model no longer identifies a reliable direction of improvement. In least-squares problems, the analogous issue often appears through an ill-conditioned Jacobian or normal matrix $J^T J$, which can destabilize both parameter updates and uncertainty estimates in routines such as `least_squares`.

8.1. Consequences

Singular Hessians may produce:

- Newton step divergence
- Matrix inversion failure
- Unstable covariance estimates
- Flat optimization directions

8.2. Regularization

A common mitigation strategy is:

$$H_{\text{reg}} = H + \lambda I \tag{9}$$

where λ is a damping coefficient. This modification increases curvature in weakly identified directions and reduces the risk of taking excessively large Newton steps.

9. Sensitivity to Initialization

Curvature and scaling problems become especially important in nonconvex landscapes, where many optimization algorithms are highly sensitive to starting conditions. Different initializations may converge toward entirely different minima, even when the same algorithm and tolerances are used.

9.1. Nonconvex Optimization

Nonconvex landscapes contain:

- Local minima
- Saddle points
- Flat plateaus
- Chaotic gradient regions

9.2. Scientific Consequences

In physical simulations, sensitivity to initialization can lead to inconsistent parameter estimation across repeated experiments. A robust workflow should therefore report not only the best solution found, but also how that solution changes across plausible starting points, preprocessing choices, and optimizer configurations.

10. Existing Tooling and Diagnostics

Instability can arise from objective geometry, parameterization, arithmetic, derivative estimation, or initialization. The following diagnostics make these mechanisms visible during an optimization run rather than only after an unexpected result has already been accepted.

10.1. Condition Number Analysis

Tracking condition numbers can reveal instability early. In least-squares problems, the conditioning of the Jacobian or normal equations indicates whether the data identify the fitted parameters. A rapidly increasing condition number may signal redundant parameters, insufficient excitation in the data, or a poor parameterization.

10.2. Gradient Norm Monitoring

Gradient norms should generally decrease as the optimizer approaches a stationary point. Sudden spikes, long plateaus, or oscillations in the gradient norm may indicate finite-difference noise, poor scaling, overly aggressive line-search steps, or an objective function that is not smooth at the current numerical resolution.

10.3. Sensitivity Analysis

Perturbing inputs slightly and observing output variation helps evaluate robustness. Useful perturbations include changing the initial condition, adding small noise to data, modifying tolerances, and switching between comparable algorithms. If these changes alter the scientific conclusion, the optimization result should be treated as fragile.

10.4. Eigenvalue Analysis

The Hessian eigenvalue spectrum provides insight into optimization geometry. Large eigenvalue ratios indicate anisotropic curvature, near-zero eigenvalues indicate flat or weakly identifiable directions, and negative eigenvalues reveal that the current point is not a strict local minimum.

10.5. Convergence Logs

Convergence logs often reveal instability that scalar termination messages hide. Objective values, step sizes, residual norms, gradient norms, warnings, and parameter summaries can expose stagnation, cycling, or abrupt jumps that are difficult to infer from the final optimizer message alone. These diagnostics motivate the experimental summaries in the next section.

10.6. Relation to Existing Diagnostic Tooling

Many of these diagnostics are already supported by mature scientific software. SciPy optimization routines expose convergence messages, status codes, callback hooks, derivative options, Hessian or Hessian-vector interfaces for second-order methods, and trust-region algorithms whose behavior can be inspected through iteration counts, function evaluations, gradient norms, and termination messages [6]. These features provide low-level access to optimizer behavior.

Derivative and curvature tooling further expands this diagnostic ecosystem. Packages such as `numdifftools` can approximate gradients, Jacobians, and Hessians by finite differences, supporting derivative checks and sensitivity analysis when analytic derivatives are unavailable [7]. Automatic differentiation systems such as JAX can provide programmatic gradients, Jacobians, Hessians, and gradient-consistency checks when the objective is differentiable and written in a compatible computational model [8]. In machine-learning workflows, libraries such as Optax provide optimizer-state monitoring, adaptive gradient transformations, learning-rate schedules, and gradient clipping mechanisms that help control unstable updates [9].

The present work does not replace these tools. It organizes their mechanisms—derivative evaluation, callbacks, curvature inspection, trust-region control, and optimizer-state moni-

toring—into a workflow-level framework for identifying and reporting instability patterns across scientific optimization pipelines.

11. Large-Scale Optimization Reliability Study

The empirical study is not intended to establish universal optimizer rankings. Its purpose is to characterize optimization reliability under realistic conditioning and preprocessing regimes using large-scale scientific data. The central question is whether optimizer choice dominates reliability, or whether preprocessing and conditioning substantially reduce optimizer sensitivity.

11.1. UCI HIGGS Dataset

The main experiment uses the HIGGS dataset (UCI Machine Learning Repository dataset 280, DOI: D. Whiteson [10]) [10]. From the repository's `HIGGS.csv.gz` release, the study samples 250,000 of the 11,000,000 simulated events, each containing 28 real-valued physics features and a binary classification target. The optimization problem is logistic regression, solved through SciPy optimizers across 100 randomized train/test splits. The complete HIGGS pipeline and instability stress test are archived with the paper as a supplementary Jupyter notebook. Logistic regression was selected because it provides a smooth and interpretable optimization landscape whose conditioning behavior can be analyzed directly without introducing deep-learning-specific optimizer dynamics [11]. This setting combines a real scientific dataset, moderate-dimensional feature geometry, and a smooth objective whose Hessian conditioning can be measured directly.

The evaluated methods are BFGS, CG, L-BFGS-B, and Powell. Powell was retained because derivative-free optimization remains common in scientific workflows where stable analytic derivatives are unavailable. Each method is run with and without feature scaling so that optimizer behavior can be compared under different conditioning regimes. The reported metrics include success rate, test log loss, test accuracy, Hessian condition number, function evaluations, iterations, and runtime. These HIGGS results are the core empirical evidence in the paper; the earlier analytic examples serve only to explain mechanisms.

Table 3. *Optimization reliability on the UCI HIGGS dataset across 100 randomized train/test splits.*

Scaling	Method	Success Rate	Test LogLoss	Test Accuracy	Mean Hessian Condition	Mean Runtime (s)
Unscaled	BFGS	1.00	0.640348	0.634407	1712.31	4.09
Unscaled	CG	1.00	0.640350	0.634393	1712.28	9.47
Unscaled	L-BFGS-B	1.00	0.640347	0.634400	1712.31	4.69
Unscaled	Powell	1.00	0.646431	0.621087	1686.90	93.99
Scaled	BFGS	1.00	0.638777	0.639535	54.29	1.68
Scaled	CG	1.00	0.638777	0.639529	54.29	2.69
Scaled	L-BFGS-B	1.00	0.638777	0.639538	54.29	1.34
Scaled	Powell	1.00	0.638806	0.639465	54.30	58.27

Table 4. *Computational cost comparison across optimization methods on the HIGGS experiment.*

Scaling	Method	Mean Iterations	Function Evaluations	Runtime (s)
Unscaled	BFGS	99.48	101.43	4.09
Unscaled	CG	122.52	235.15	9.47

Unscaled	L-BFGS-B	65.98	75.56	4.69
Unscaled	Powell	18.66	5057.94	93.99
Scaled	BFGS	40.07	41.07	1.68
Scaled	CG	33.55	69.28	2.69
Scaled	L-BFGS-B	19.47	22.67	1.34
Scaled	Powell	11.10	3116.34	58.27

Table 5. *Impact of preprocessing and scaling on optimization reliability metrics.*

Metric	Unscaled Mean	Scaled Mean	Relative Improvement
Hessian Condition Number	1712.30	54.29	31.5× reduction
Mean Runtime	28.06	15.00	46.5% reduction
Mean Function Evaluations	1367.52	812.35	40.6% reduction
Mean Test Accuracy	0.6311	0.6395	Improved convergence consistency

11.2. Statistical Effect of Scaling

To evaluate whether the scaling improvements reflect consistent behavior rather than isolated averages, we compared paired scaled and unscaled runs across randomized train/test splits. For each optimizer, the scaled and unscaled configurations were matched by seed, allowing direct comparison of preprocessing effects on the same data partition.

Table 6 reports the relative change in Hessian conditioning, function evaluations, and runtime after scaling. The runtime and conditioning improvements after scaling were consistent across randomized splits and substantially exceeded the observed run-to-run variance. Paired Wilcoxon signed-rank tests comparing scaled and unscaled runtime distributions yielded $p < 0.001$ across all gradient-based methods.

Table 6. *Effect of feature scaling on Hessian conditioning, function evaluations, and runtime in the HIGGS optimization experiment.*

Method	Condition Reduction	Function Eval. Reduction	Runtime Reduction
BFGS	31.5 ×	59.5%	59.0%
CG	31.5 ×	70.5%	71.6%
L-BFGS-B	31.5 ×	70.0%	71.3%
Powell	31.0 ×	38.4%	38.0%

The reduction is especially pronounced for gradient-based methods. After scaling, BFGS, CG, and L-BFGS-B converge to nearly identical predictive performance, but require substantially fewer iterations and function evaluations. Powell is also stable after scaling, but remains substantially slower and slightly worse in test log loss and accuracy, underscoring the robustness-at-high-cost tradeoff even in this moderate-dimensional setting. Derivative-free methods demonstrated robustness advantages in certain low-dimensional instability regimes, but incurred substantially larger computational costs on higher-dimensional scientific optimization tasks.

11.3. Conditioning Dominates Optimizer Differences

After scaling, BFGS, CG, and L-BFGS-B produce nearly identical log loss, accuracy, and Hessian conditioning. Runtime and evaluation-count differences also shrink substantially. The Hessian condition number falls from approximately 1712 to approximately 54, a more

than order-of-magnitude improvement. In this experiment, the remaining method differences are best interpreted as robustness-efficiency tradeoffs rather than evidence for a universally superior optimizer.

11.4. Controlled Instability Stress Test

To evaluate whether the auditing framework can detect genuine instability rather than only mild conditioning effects, a controlled instability stress test was performed on the HIGGS optimization workflow. The goal was to intentionally induce numerical failure modes while preserving the underlying scientific dataset and optimization task.

The instability regime was constructed using three controlled perturbations. First, feature-scale imbalance was introduced using multiplicative scaling factors logarithmically spaced between 10^{-8} and 10^{8}. Second, near-collinear structure was induced by replacing selected features with linear combinations satisfying pairwise correlation coefficients exceeding 0.999999. Third, additive Gaussian perturbation noise with standard deviation 10^{-3} was injected into the feature matrix. These modifications were designed to create severe Hessian conditioning, unstable gradient geometry, and optimizer-status reliability mismatches. The resulting optimization behavior was then compared against a scaled mitigation pipeline using the same randomized train/test splits and optimizer configurations.

Practical success was defined using three simultaneous criteria: (1) successful optimizer termination, (2) finite Hessian conditioning below 10^{12}, and (3) gradient norm below 10^{-3}. The practical-success criteria were intentionally conservative and designed to identify early numerical degradation rather than only catastrophic optimizer failure.

Table 7. *Controlled instability stress test on the HIGGS optimization workflow. Practical success required successful optimizer termination, finite Hessian conditioning below 10^{12}, and gradient norm below 10^{-3}.*

Variant	Method	Optimizer Success	Practical Success	False Success	Median Hessian Condition	Mean Gradient Norm
Scaled	BFGS	1.00	1.00	0.00	5.1×10^{1}	1.7×10^{-5}
Scaled	CG	1.00	1.00	0.00	5.1×10^{1}	1.4×10^{-5}
Scaled	L-BFGS-B	1.00	1.00	0.00	5.1×10^{1}	1.3×10^{-5}
Scaled	Powell	1.00	0.00	1.00	5.0×10^{1}	3.3×10^{-3}
Unstable	BFGS	0.00	0.00	0.00	∞	4.9×10^{7}
Unstable	CG	0.00	0.00	0.00	∞	4.9×10^{7}
Unstable	L-BFGS-B	0.00	0.00	0.00	∞	4.8×10^{7}
Unstable	Powell	1.00	0.00	1.00	1.1×10^{38}	2.0×10^{7}

To evaluate instability emergence rather than only catastrophic failure, additional scale-imbalance ablations were performed across progressively more severe perturbation regimes. The severity ablation varied feature-scale imbalance while holding additive perturbation noise and near-collinear feature construction fixed at the same levels used in the full instability stress test. The results demonstrated a gradual transition from stable convergence to widespread practical failure as Hessian conditioning increased, indicating that the auditing framework can detect incipient instability before complete optimizer breakdown occurs.

Table 8. *Instability-emergence ablation across progressively severe scale-imbalance regimes in the HIGGS optimization workflow.*

Severity	Scale Range	Practical Success Rate	False Success Rate	Median Hessian Condition	Mean Gradient Norm
Moderate	10^{-2} to 10^{2}	0.333	0.000	4.704×10^{6}	1.67×10^{-2}
Severe	10^{-4} to 10^{4}	0.333	0.003	3.062×10^{10}	8.02×10^{0}
Extreme	10^{-8} to 10^{8}	0.000	0.187	1.053×10^{18}	1.864×10^{5}

The severity-ablation results demonstrate that instability emerges progressively rather than abruptly. As scale imbalance increased from moderate to extreme regimes, Hessian conditioning and gradient norms increased by multiple orders of magnitude while optimizer-status reliability deteriorated substantially. False-success rates also increased under severe perturbation, indicating that optimizer termination criteria became progressively less reliable as numerical conditioning worsened. These results suggest that workflow-level auditing can detect instability emergence prior to complete optimizer breakdown.

Notably, even moderate scale imbalance produced substantial practical failure rates despite relatively stable predictive performance. Inspection of individual runs showed that CG and L-BFGS-B frequently failed gradient-based practical-success criteria under moderate conditioning growth, suggesting that workflow-level auditing can detect incipient numerical degradation prior to catastrophic optimizer collapse.

The similar practical-success rates observed under moderate and severe perturbation were primarily driven by repeated failure of the same optimizer classes, particularly CG and L-BFGS-B, under increasingly ill-conditioned geometry, while BFGS retained practical success in the majority of moderate-severity runs. Although practical-success frequencies remained numerically similar across these regimes, gradient norms and Hessian conditioning deteriorated by multiple additional orders of magnitude under severe perturbation, indicating substantially greater underlying numerical instability.

False-success behavior emerged primarily in derivative-free optimization trajectories where optimizer termination criteria were satisfied despite persistent gradient instability. The substantial increase in false-success rate under extreme perturbation indicates that optimizer-status reliability deteriorates progressively as conditioning pathologies intensify.

An especially notable result is the behavior of Powell's method under the scaled regime. Although optimizer termination was consistently reported as successful, the method failed practical-success criteria because gradient norms remained above the operational stability threshold of 10^{-3}. In contrast, gradient-based methods converged to norms near 10^{-5}. Powell therefore terminated in regions that satisfied internal convergence conditions while remaining substantially farther from stable local stationary behavior. This indicates that derivative-free convergence criteria may not adequately reflect local numerical stability even after preprocessing mitigates Hessian conditioning.

The instability stress test demonstrates that optimizer termination messages alone are insufficient indicators of numerical reliability. Under induced instability, all gradient-based methods failed practical-success criteria and exhibited catastrophic Hessian conditioning together with gradient-norm explosion. Powell reported successful optimizer termination despite failing all practical-success criteria, producing a clear optimizer-status reliability mismatch.

After scaling and mitigation, Hessian conditioning returned to stable levels and gradient norms decreased by more than twelve orders of magnitude. The experiment therefore directly instantiates several instability categories from the earlier taxonomy, including geometric instability, differential instability, and structural instability.

11.5. Optimizer-Status Reliability Mismatches

A *false success* occurs when an optimizer reports successful convergence while remaining substantially distant from a known global optimum or physically meaningful solution. A *false failure* occurs when an optimizer reports unsuccessful termination despite satisfying objective-value or residual criteria. These status-reliability mismatches motivate reporting objective values, diagnostics, and sensitivity checks alongside optimizer messages. The instability stress test in Table 7 demonstrates that these mismatches occur empirically even in standard scientific machine-learning workflows when severe conditioning pathologies are introduced.

11.6. Scope and Limitations

The HIGGS experiment studies moderate-dimensional smooth optimization through logistic regression and does not capture the optimization geometry of deep neural networks or extremely high-dimensional nonconvex objectives. The conclusions should therefore be interpreted as workflow-level observations for classical scientific optimization pipelines rather than universal statements about all optimization systems.

12. Numerical Auditing Framework

The mitigation strategies can be organized as a staged protocol that makes existing diagnostics systematic, reproducible, and interpretable at the workflow level. Existing tools provide local measurements; the purpose here is to assemble them into a reliability audit for scientific optimization workflows.

Table 9. *Workflow-oriented numerical auditing protocol for scientific optimization pipelines.*

Audit Stage	Diagnostic Actions	Purpose
Pre-Optimization Audit	Feature scaling, condition estimation, derivative verification	Reduce instability before optimization
Runtime Audit	Gradient monitoring, convergence checks, Hessian inspection	Detect unstable optimization dynamics
Post-Optimization Audit	Sensitivity analysis, multi-start validation, residual inspection	Validate reliability of obtained solutions

Although instability thresholds are problem-dependent, the experiments in this work suggest several practical heuristics. In the HIGGS stress-test regime, Hessian condition numbers exceeding approximately 10^8 were consistently associated with severe optimizer degradation and gradient explosion. Stable optimization regimes typically converged to gradient norms near 10^{-5}, whereas unstable or practically unreliable runs remained above 10^{-3}. Large increases in function-evaluation count without corresponding gradient reduction also frequently preceded optimizer-status reliability mismatches.

12.1. Stage 1: Pre-Optimization Audit

Before running the optimizer, the model formulation should be checked for numerical risk. This stage includes scale analysis, nondimensionalization or normalization of parameters, inspection of parameter bounds, condition estimation for available Jacobians or linearized subproblems, and derivative verification using analytic, automatic-differentiation, complex-step, or finite-difference checks.

12.2. Stage 2: Runtime Audit

During optimization, the workflow should log objective values, step sizes, gradient norms, constraint violations, condition estimates, callback traces, Hessian or Hessian-vector diagnostics when available, optimizer status messages, iteration counts, function evaluations, and warning messages.

12.3. Stage 3: Post-Optimization Audit

After termination, the returned solution should be tested through perturbation sensitivity analysis, multi-start validation, comparison across algorithm families when feasible, residual and gradient checks, Hessian spectrum or rank inspection, and physical plausibility checks. A complete audit report should include both numerical diagnostics and domain-specific validity checks.

13. A Taxonomy of Numerical Stability Failures

The empirical results and mechanistic examples can be organized into four recurring failure types. The taxonomy is intended as a diagnostic map: it connects observed optimizer behavior to likely numerical causes and first-line mitigations.

Table 10. *Taxonomy of numerical stability failures in scientific optimization pipelines.*

Failure Type	Cause	Observable Symptoms	Mitigation Strategy
Geometric Instability	Ill-conditioning, curved valleys, anisotropic curvature	Slow convergence, oscillation, sensitivity to initialization	Feature scaling, preconditioning, trust-region methods
Arithmetic Instability	Overflow, underflow, catastrophic cancellation	NaNs, infinities, unstable finite differences	Stable formulations, clipping, higher precision
Differential Instability	Noisy gradients, inaccurate derivatives	Gradient oscillation, premature convergence, unstable updates	Autodiff, smoothing, gradient verification
Structural Instability	Singular Hessians, rank deficiency	Optimization stagnation, unstable inverse solutions	Regularization, pseudoinverse methods, dimensionality reduction

Geometric instability is most directly addressed by scaling, preconditioning, and trust-region control. Arithmetic instability calls for stable reformulation and attention to finite-precision effects. Differential instability requires more reliable derivative handling through analytic derivatives, automatic differentiation, smoothing, or derivative checks. Structural instability indicates weak identifiability or rank deficiency and often requires regularization, constraints, or dimensionality reduction.

14. Mitigation Strategies

The statistical experiments and taxonomy suggest that stability is not achieved by choosing a single “best” optimizer. The safeguards below map the failure taxonomy to practical interventions.

14.1. Normalization

Parameters should be scaled before optimization so that comparable changes in different coordinates produce comparable changes in the objective. In physical models, nondimensionalization is often preferable to purely statistical scaling because it preserves scientific interpretation while reducing numerical anisotropy.

14.2. Analytic Gradients

Analytic derivatives reduce finite-difference instability and often improve both accuracy and speed. When exact analytic gradients are unavailable, automatic differentiation, complex-step differentiation, or carefully validated finite-difference steps may provide more reliable alternatives than default numerical differencing.

14.3. Regularization

Regularization improves conditioning by penalizing unstable directions in parameter space. In inverse problems and curve fitting, ridge penalties, prior constraints, or damping terms can reduce variance and prevent the optimizer from exploiting noise in poorly identifiable directions.

14.4. Trust-Region Methods

Trust-region approaches improve stability under severe curvature imbalance by limiting each step to a region where the local model is expected to be reliable. This is especially useful when Newton or quasi-Newton steps point toward directions dominated by noisy curvature estimates.

14.5. Multiple Initializations

Optimization should be repeated from multiple starting points. Multi-start experiments help separate stable solutions from artifacts of initialization and provide practical evidence about the presence of local minima.

14.6. Constraint Enforcement

Physical constraints can stabilize optimization by excluding meaningless regions of parameter space. Bounds, positivity transformations, conservation laws, and monotonicity constraints can prevent overflow, singular model evaluations, and nonphysical fitted parameters.

15. Conclusion

Optimization reliability depends strongly on conditioning, preprocessing, and numerical auditing. Across the HIGGS experiments, feature scaling reduced the mean Hessian condition number from approximately 1712 to approximately 54, reduced function evaluations and runtime, and made the standard gradient-based optimizers behave nearly identically in predictive performance.

The controlled instability stress test further demonstrated that severe conditioning pathologies can produce optimizer-status reliability mismatches, gradient explosions, and effectively singular Hessian structure even on standard scientific datasets. These failures became detectable through workflow-level auditing metrics including condition estimates, gradient norms, and practical-success validation criteria.

These results support the paper's central claim: workflow-level numerical auditing may be more important than optimizer selection alone in many scientific computing pipelines.

Optimizer choice still matters, especially for computational cost and derivative-free scalability, but the strongest reliability gains in the large-scale experiment came from improving the numerical formulation before and around the optimizer.

References

[1] J. Nocedal and S. J. Wright, *Numerical Optimization*. Springer, 2006. doi: 10.1007/978-0-387-40065-5.

[2] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. SIAM, 2002. doi: 10.1137/1.9780898718027.

[3] G. H. Golub and C. F. Van Loan, *Matrix Computations*. Johns Hopkins University Press, 2013. doi: 10.56021/9781421407944.

[4] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*. SIAM, 1997. doi: 10.1137/1.9780898719574.

[5] W. H. Press, S. A. Teukolsky, W. T. Vetterling, and B. P. Flannery, *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, 2007.

[6] P. Virtanen *et al.*, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[7] P. A. Brodtkorb and J. D'Errico, "Numdifftools: Solving Automatic Numerical Differentiation Problems in Python." [Online]. Available: https://github.com/pbrod/numdifftools

[8] J. Bradbury *et al.*, "JAX: Composable Transformations of Python+NumPy Programs." [Online]. Available: https://github.com/jax-ml/jax

[9] DeepMind, "Optax: Gradient Processing and Optimization in JAX." [Online]. Available: https://github.com/google-deepmind/optax

[10] D. Whiteson, "HIGGS [Dataset]." [Online]. Available: https://archive.ics.uci.edu/dataset/280/higgs

[11] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004. doi: 10.1017/CBO9780511804441.